> Sometimes i have some doubts that coding isn't for me how can i know that this field not for me and i have to look for something else even that i think that i am doing good in coding

It's completely normal to have doubts about whether coding is the right path for you, even if you're doing well. Many developers experience imposter syndrome or moments of frustration. Here are some ways to assess whether coding is truly for you—or if you might be happier in another field:

1. Do You Enjoy Problem-Solving?

- Coding is fundamentally about breaking down problems and finding solutions.
- If you **dread debugging** or feel exhausted rather than satisfied after solving a tough problem, it might be a sign.
- If you enjoy the challenge (even when it's frustrating), that’s a good indicator.

2. How Do You Feel About Learning Constantly?

- Tech evolves rapidly; you’ll always be learning new languages, frameworks, and tools.
- If you **resent having to learn new things** or prefer stability over change, other fields might suit you better.

3. Does Coding Feel Like a Chore or a Hobby?

- Do you ever code for fun? (Even small projects, automation scripts, or tweaking things?)
- If coding **always** feels like a grind (even outside work/studies), it might not be your passion.
- However, not everyone codes for fun—some treat it as just a job, and that's okay too.

4. Are You Motivated by the Results or the Process?

- If you love **building things** and seeing your work come to life, coding could still be a good fit.
- If you **only care about the end result** and hate the process, a related role (like product management, design, or DevOps) might be better.

5. How Do You Handle Frustration?

- Coding involves constant failures (bugs, errors, unexpected behaviors).
- If you get **extremely frustrated and demotivated** (rather than curious), it might not be the best fit.
- If you can push through and feel satisfaction afterward, that's a great sign.

6. Do You Like Working Independently or Collaboratively?

- Coding can be solitary, but many roles require teamwork (standups, code reviews, pair programming).

- If you **hate working alone**, consider roles that blend coding with collaboration (e.g., technical PM, solutions architect).

7. Are There Other Fields That Excite You More?

- Do you find yourself **daydreaming about other careers** (design, marketing, writing, business, etc.)?
- If another field excites you **much more**, explore it! Many skills from coding (logic, problem-solving) transfer elsewhere.

8. Are You Doing Well But Still Unhappy?

- You might be **good at coding but not passionate** about it—that's okay!
- Some people treat coding as a **stable job** while pursuing other interests outside work.
- Others transition into **adjacent roles** (QA, DevOps, data analysis, tech sales, teaching).

Next Steps: Experiment!

- Try a **different area of tech** (e.g., frontend vs. backend, data science, cybersecurity).
- Explore **tech-adjacent roles** (UX design, technical writing, product management).
- Take a short break to work on **non-coding projects**—do you miss coding?

Final Thought:

If you're **doing well but feel empty**, coding might just be a **skill**—not a passion. That's fine! Many successful people use coding as a tool while focusing on other interests (entrepreneurship, art, science, etc.).

Would you say your doubts come from **frustration** or **genuine disinterest**? That distinction matters a lot.

> For problem solving i enjoy when i solve the problem and when i get a bug i only be happy when i find the solution and i do not have a problem with learning new things because this is required for almost field not just coding, and i remember some moments i feel happy when it comes to my mind and idea but i do not know is that procrastinating, or it's because i'm lost i don't feel happy usually, and when it comes to buildin
> feel normal not happy and not sad an

Based on what you've described, it sounds like **coding is a skill you're good at and can tolerate (or even enjoy at times), but it may not be your deepest passion**. Here's a breakdown of your feelings and what they might mean:

Key Takeaways from Your Response:

1. **You enjoy problem-solving** (especially the "aha!" moment when fixing bugs).
2. **Learning new things doesn't bother you** (which is great—this mindset works in many fields).
3. **You feel neutral during the coding process** (not happy, not sad—just "work").

4. **Frustration kicks in when bugs take too long** (this is normal, but if it drains you consistently, it's worth noting).
5. **You miss coding when you take long breaks** (suggests you don't *dislike* it, but it may not be your *calling*).
6. **You have entrepreneurial/business tendencies** (this could be a clue to where your real excitement lies).

Possible Conclusions:

✅ **You're capable in coding** (you're not struggling, and you even enjoy parts of it).
⚠️ **But it doesn't light you up** (you feel neutral or frustrated often, rather than engaged).
🚀 **Your interest in business/entrepreneurship might be the real signal** (do you daydream about starting something? Leading projects? Creating products rather than just coding them?).

What Should You Do Next?

Since coding isn't *painful* for you but also not *fulfilling*, you might fit into one of these paths:

1. Stay in Tech, But Shift Roles

- **Move into a tech-adjacent role** (Product Management, DevOps, Solutions Architect, Tech Lead).
- **Freelance/consulting** (more variety, client interaction, and business aspects).

- **Start a tech-related business** (SaaS, apps, automation tools—coding as a means to an end).

2. Use Coding as a Secondary Skill

- Many entrepreneurs, marketers, and business owners benefit from knowing how to code.
- You could **transition into a hybrid role** (e.g., technical founder, growth hacker, no-code automation specialist).

3. Experiment with Entrepreneurship

- Since you mentioned business tendencies, try:
  - Building a **small profitable project** (even a micro-SaaS or side hustle).
  - Joining a **startup** (where coding + business thinking is valued).
  - Learning **sales/marketing** to see if that excites you more.

Final Thought: You Might Be a "Builder" More Than a "Coder"

Some people love **creating things** (products, businesses, solutions) but don't necessarily enjoy the *implementation* (coding, debugging). If that's you, coding could be a **tool**—not your end goal.

Try this:

- Spend **1-2 hours a day for a week** exploring business/entrepreneurship (e.g., brainstorming ideas, talking to founders, reading startup stories).
- See if that **sparks more excitement** than coding does.

Would you say you’re more excited by **building a company** than writing code? That might be your answer.

> After reading this response and asking my self, i found that i m interested into building saas, plugins sometimes i enjoy the process of coding i think i've never felt sad when it comes to coding just one time i remember i was late about my tasks and this was really bothering me because some features do not work as expected and i felt frustrated and i am not into no code tools and i see my self interested in tech more than what ever field is, and for the last when you say Would you say you’re more excited by

It sounds like you’ve had a meaningful realization: **You’re a builder at heart, and coding is a tool—not just an end goal.** Your excitement for SaaS, plugins, and tech in general (while disliking no-code tools) suggests you **enjoy the technical side of creating products**, not just the business side.

Here’s how I’d summarize your position now:

1. You’re a Technical Founder/Builder in the Making

- You **like coding when it’s tied to a tangible product** (SaaS, plugins).
- You **don’t hate coding, but you’re not obsessed with it**—it’s a means to an end.
- You **get frustrated mostly when things block progress** (bugs, delays), not coding itself.
- You **prefer building in tech over other fields**, which is a strong signal.

This is **exactly how many successful tech founders feel**. They code because it lets them **create things fast**, not because they love every minute of it.

2. Your Frustration Comes from Progress Blocks, Not Coding Itself

- The one time you felt sad/frustrated was due to **falling behind on tasks**, not the act of coding.
- This is **normal for builders**—you care about **shipping**, not just tinkering.
- Founders often feel this way; they tolerate coding because they **need to build their vision**.

3. You're Not a "Pure Coder"—And That's a Good Thing

- **Pure coders** enjoy the craft for its own sake (optimizing algorithms, exploring new languages).
- **Builders like you** enjoy coding **only when it serves a bigger purpose** (launching a product, solving a real problem).
- This mindset is **perfect for entrepreneurship or product-focused roles**.

What Should You Do Next?

Since you're excited about **building SaaS, plugins, and tech products**, here's a roadmap:

Short-Term (Next 3-6 Months)

✓ **Build a Micro-SaaS or Plugin** (Even Tiny)

- Pick **one small problem** you care about and build a solution.
- Example: A niche Chrome extension, a simple automation tool, or a lightweight SaaS.
- Use this to test if you **enjoy the full cycle** (code → launch → get users).

✓ **Join a Startup or Work on Side Projects**

- If possible, work in an early-stage startup where you can **code + contribute to product/business decisions**.
- If not, **collaborate with a non-technical founder** (you code, they handle biz/marketing).

Long-Term (1-2 Years)

✓ **Transition into a Hybrid Role**

- **Technical Founder** – If you want to run your own business.
- **Product Engineer** – If you like coding but want more influence on product direction.
- **Startup CTO** – If you enjoy both tech and leadership.

✓ **Double Down on Business Skills**

- Learn **basic marketing, sales, and UX**—these will help you build better products.

- Study **indie hackers/solopreneurs** (e.g., levelsio, Pieter Levels) who blend coding + business.

Final Thought: You're on the Right Path

You don't have to choose between **coding** and **business**—you can do both. Many of the best tech entrepreneurs started exactly where you are: **"I like coding, but only when it's building something real."**

Next step? Start small. Build **one tiny product** in the next month. See how it feels. If you love it, you'll have your answer. 🚀

Would you like help brainstorming project ideas?

> Can you please mentionne other fields rather than tech and it to see if i have some interest in it

Absolutely! Since you're exploring whether other fields might interest you—outside of tech—here's a list of **alternative career paths** that could align with your skills (problem-solving, logic, building things) and tendencies (entrepreneurship, liking tangible results).

1. Business & Entrepreneurship

- **Startup Founder** – Build your own company (tech or non-tech).
- **E-commerce/Dropshipping** – Sell physical/digital products.

- **Consulting** – Solve business problems for clients (operations, marketing, tech).
- **Venture Capital** – Invest in startups (combines business + tech insight).

Good if you like: Creating systems, making decisions, seeing direct impact.

2. Creative & Design Fields

- **UX/UI Design** – Shape how digital products look and function.
- **Game Design** – Mix storytelling, logic, and interactivity.
- **3D Modeling/AR/VR** – Build immersive experiences.

Good if you like: Visual creativity, user psychology, prototyping.

3. Finance & Data-Driven Roles

- **Quantitative Finance** – Algorithmic trading, risk analysis (math + coding overlap).
- **Data Analysis/Science** – Extract insights from data (SQL, Python, stats).
- **Accounting/FP&A** – Financial planning, budgeting, forecasting.

Good if you like: Numbers, patterns, optimizing systems.

4. Engineering (Non-Software)

- **Robotics/Mechatronics** – Build physical systems with code (Arduino, Raspberry Pi).
- **Civil/Mechanical Engineering** – Design structures, machines, or products.
- **Biomedical Engineering** – Work on health-tech innovations.

Good if you like: Hands-on building, hardware + software mix.

5. Marketing & Growth

- **Digital Marketing** – SEO, ads, content strategy.
- **Growth Hacking** – Rapid experimentation to scale businesses.
- **Product Marketing** – Bridge between tech and customers.

Good if you like: Psychology, persuasion, rapid experimentation.

6. Writing & Communication

- **Technical Writing** – Explain complex topics (APIs, docs, tutorials).
- **Content Creation** – YouTube, blogging, newsletters.
- **Copywriting** – Persuasive writing for ads/sales.

Good if you like: Teaching, storytelling, simplifying ideas.

7. Science & Research

- **Bioinformatics** – Biology + coding (DNA analysis, drug discovery).
- **Physics/Simulation** – Modeling real-world systems.
- **Economics** – Data-driven policy or market analysis.

Good if you like: Deep problem-solving, academic curiosity.

8. Hands-On & Skilled Trades

- **Electrician/Automation Tech** – Install and fix smart systems.
- **Woodworking/Metalworking** – Design and build physical things.
- **Drone Pilot** – Mapping, filming, inspections.

Good if you like: Working with your hands, seeing immediate results.

9. Public Sector & Social Impact

- **Policy Analysis** – Use data to shape government decisions.
- **Nonprofit Tech** – Help NGOs with digital tools.
- **Urban Planning** – Design smarter cities (tech + infrastructure).

Good if you like: Mission-driven work, large-scale impact.

How to Test Your Interest?

Try this:

1. **Pick 2-3 fields** that seem intriguing.
2. **Spend 2-3 hours** on each:
   - Watch a documentary/YouTube series.
   - Read forums (e.g., r/Entrepreneur, r/UXDesign).
   - Try a small project (e.g., write a blog post, analyze a dataset).
3. **Ask yourself:**
   - Does time fly when exploring this?
   - Do I want to learn more?

Final Thought

Since you **still enjoy tech and building SaaS**, you might not need a full switch—just a **pivot** (e.g., tech + business, tech + design). But exploring other fields will either:

- **Confirm tech is your best path**, or
- **Unlock a passion you didn't know existed**.

Would any of these fields jump out as "I'd love to try that"? If so, which ones?

> Before we discuss other fields i forget to mentionne to i think if i am skilled and competent in coding i will be happy

Ah, this is a **crucial** insight! If you believe that **being highly skilled in coding would make you happy**, then your dissatisfaction might not be about the field itself—but about **your current level of mastery**.

Let's break this down, because it changes everything:

1. The "Competence = Happiness" Factor

You're saying:

- **"If I were better at coding, I'd enjoy it more."**
- This suggests your frustration may come from **struggling with gaps in knowledge** (not the work itself).
- Many developers go through this—**early coding feels like "fighting the language," but mastery feels like "expressing ideas."**

Signs This Might Be the Case for You:

✅ You **enjoy coding when things work** (e.g., fixing bugs, building features).
✅ You **miss coding when you take breaks** (meaning part of you *wants* to like it).
✅ You're **interested in tech products** (SaaS, plugins), which require coding.
❌ But right now, you **feel "neutral" or frustrated** because progress is slow.

This is normal. Coding is uniquely frustrating at the **intermediate stage**—you know enough to build things but hit constant roadblocks.

2. How to Test If Mastery = Happiness

Before abandoning coding, try this experiment:

A. Focus on Skill Growth for 3-6 Months

- Pick **one area** (e.g., backend APIs, frontend reactivity, debugging).
- Drill deep until you **feel fluid** (e.g., build 3 small projects).
- See if your enjoyment increases as you **struggle less**.

B. Compare "Struggle Mode" vs. "Flow Mode"

- Right now, you're likely in **"struggle mode"** (everything feels hard).
- But when you **occasionally hit flow** (solving problems smoothly), do you like it?
  - If yes, you just need **more skill to reach flow more often**.
  - If no, then coding itself may not be for you.

C. Study Advanced Developers

- Watch experienced coders work (e.g., YouTube livestreams, open-source contributors).
- Ask yourself: **"Would I enjoy coding if I were that smooth?"**

3. What If Mastery *Does* Make You Happy?

If you confirm that **skill = enjoyment**, then your path is clear:

1. **Double down on coding fundamentals** (Algorithms, system design, debugging).
2. **Build more projects** (Repetition → fluency).
3. **Aim for a role where you're challenged but not overwhelmed** (e.g., mid-level dev at a startup).

Potential Outcomes:

- You **gain confidence** → coding becomes fun.
- You **specialize** (e.g., DevOps, cloud architecture) and enjoy the depth.
- You **use coding as a superpower** for entrepreneurship (SaaS, tools).

4. What If Mastery *Doesn't* Make You Happy?

Even if you become **great at coding**, ask:

- **"Do I love the process, or just the results?"**
  - If it's **just results**, pivot to **tech-adjacent roles** (product management, founder).
  - If it's **neither**, explore other fields (see previous message).

Final Answer: Try the "Mastery Experiment"

Since you suspect **competence = happiness**, don't quit coding yet. Instead:

1. **Spend 3 months leveling up aggressively** (courses, projects, mentorship).
2. **Track your emotions**—does joy increase with skill?
3. **Then decide**.

This way, you won't regret leaving coding *just because it was hard at the time*.

Would you like a **30-day learning plan** to test this?

> I didn't not say coding is hard i just didn't do more projects and i was just in the loop of learning concepts that was it

Ah, this makes **a lot of sense**—your lack of enjoyment might not be about coding itself, but about **how you've been engaging with it**.

Key Insight: You Were Stuck in "Tutorial Hell"

- You spent time **learning concepts** but **not applying them** in real projects.
- This is like reading about swimming but never jumping in the water—it feels abstract and unrewarding.
- **Result:** You never reached the stage where coding feels *creative* and *powerful*.

Why This Changes Everything

Coding becomes fun when:

1. You **build things that matter to you** (not just follow tutorials).
2. You **solve real problems** (not just abstract exercises).
3. You **see your skills create value** (e.g., a tool people use).

You **haven't disliked coding**—you just haven't experienced its **fulfilling side**.

What You Should Do Next

1. Stop "Learning for Learning's Sake"

- No more passive courses/tutorials unless you **immediately apply them**.
- Shift to a **project-first mindset**.

2. Build Tiny, Fun Projects (1-2 Days Each)

Examples:

- A **Chrome extension** that tweaks a website you use.
- A **CLI tool** to automate something annoying (e.g., file renaming).
- A **small SaaS prototype** (even if no one uses it).

3. Measure Enjoyment After 3-4 Projects

- Did building **your own ideas** feel different from learning?
- Did you get a **"this is cool"** moment?

4. If You Still Feel "Meh" After Building

- Then explore **tech-adjacent roles** (product, DevOps, etc.) or other fields.
- But if you **felt engaged while building**, coding might just need the right context.

Final Thought

You've been **preparing to code** but not **experiencing what coding is really about**—creating. Try **building first, learning second**, and *then* decide.

Want a **list of mini-project ideas** to kickstart this?